Lab Week 2 — PHP GET, POST, HEAD, OPTIONS

1) Mục tiêu bài Lab

Sau bài Lab này, học viên làm được các việc sau:

- Cài và kiểm tra môi trường gồm **VS Code, PHP 8.x, Composer, Git**, rồi chạy project local bằng web server tích hợp của PHP trong thư mục `public`.
- Khởi tạo một PHP project có cấu trúc rõ ràng với **Composer**, **PSR-4 autoload**, **.env**, **.env.example**, **config**, **storage/logs** và bootstrap tại `public/index.php`.
- Viết một ứng dụng PHP nhỏ theo bài toán thực tế mới: **quản lý workshop và đăng ký tham gia workshop**.
- Xử lý đúng các tình huống HTTP cơ bản: **GET, POST, HEAD, OPTIONS**, trả đúng **200, 201, 204, 405, 415, 422, 404** theo ngữ cảnh.
- Quản lý mã nguồn bằng **Git**, commit đúng tiến trình, và đẩy source code lên GitHub.

2) Các bước tiến hành

Bài Lab này có các bước tiến hành sau:

- Cài và kiểm tra môi trường
- Dựng cấu trúc dự án
- Khởi tạo Composer + autoload
- Tạo `.env`, `.env.example`, config
- Code ứng dụng workshop registration
- Chạy web server local bằng VS Code terminal
- Test HTTP bằng browser / curl / Postman
- Commit Git và đưa lên GitHub
- Kiểm tra và nộp bài

3) Mô tả bài toán

Tên bài toán

Mini Workshop Registration App

Bối cảnh

Một trung tâm đào tạo nhỏ cần một ứng dụng PHP đơn giản để:

- hiển thị danh sách workshop đang mở
- cho biết workshop còn chỗ hay đã đầy
- trả danh sách workshop qua HTTP endpoint
- nhận đăng ký tham gia workshop bằng HTTP POST
- kiểm tra dữ liệu gửi lên và trả status code đúng

Bài toán này phù hợp với Week 2 vì vẫn chưa cần database thật, có thể dùng **array dữ liệu mẫu**, nhưng đã đủ để dạy:

- bootstrap project
- Composer autoload
- `.env` và config
- controller
- request/response
- content type
- status code
- routing theo method/path

Phần HTTP của Week 2 nhấn mạnh rất rõ việc phân biệt request/response, method, headers và status code; đồng thời nhấn mạnh các lỗi rất hay gặp như trả sai mã 201/405/415/422.

Yêu cầu chức năng

Ứng dụng phải có:

1. Trang chủ `/`
2. API `GET /events` để trả danh sách workshop
3. API `HEAD /events`
4. API `POST /registrations` để nhận đăng ký bằng JSON
5. API `OPTIONS /registrations`
6. Trả đúng:
    - `200 OK` cho lấy dữ liệu
    - `201 Created` khi đăng ký thành công
    - `204 No Content` cho `OPTIONS`
    - `405 Method Not Allowed` nếu gọi sai method
    - `415 Unsupported Media Type` nếu gửi sai `Content-Type`
    - `422 Unprocessable Content` nếu dữ liệu hợp lệ về cú pháp nhưng thiếu/không đúng nghiệp vụ
    - `404 Not Found` nếu sai path

4) Cài đặt môi trường

Lab01 đã dùng hướng cài môi trường bằng **VS Code, PHP, Composer, Git**, mở terminal ngay trong VS Code, rồi chạy local server bằng `php -S localhost:8000 -t public`. Lab02 giữ đúng workflow đó, sau đó bổ sung thêm phần Composer autoload, `.env` và bootstrap project.

Công cụ cần có

- Visual Studio Code
- PHP 8.x

- Composer
- Git

Extension nên cài trong VS Code

- PHP Intelephense
- PHP Debug
- REST Client (khuyến nghị thêm để test HTTP trong VS Code)

Bước 1 — kiểm tra PHP

Mở terminal trong VS Code:

```
php -v
```

Bước 2 — kiểm tra Composer

```
composer --version
```

Bước 3 — kiểm tra Git

```
git --version
```

Bước 4 — cấu hình Git lần đầu

```
git config --global user.name "Ten Cua Ban"
git config --global user.email "ban@example.com"
git config --list
```

Bước 5 — mở project bằng VS Code

Trong VS Code:

- mở đúng thư mục project
- mở Terminal tích hợp
- dùng Source Control nếu muốn commit bằng giao diện

5) Tạo cấu trúc dự án

Lab01 dùng cấu trúc có `public/`, `src/`, `views/`, `composer.json`. Sang Week 2, khung project được mở rộng thêm `config/`, `storage/logs/`, PSR-4 autoload và `.env`. Lab02 dưới đây giữ tinh thần đó nhưng áp dụng cho bài toán mới.

Tạo cấu trúc như sau:

```
php-mini-workshop/
│
├── public/
│   └── index.php
│
├── src/
│   ├── Controllers/
│   │   ├── HomeController.php
│   │   ├── EventController.php
│   │   └── RegistrationController.php
│   ├── Data/
│   │   └── events.php
│   └── Support/
│       ├── Env.php
│       └── Response.php
│
├── views/
│   └── home.php
│
├── config/
│   └── app.php
│
├── storage/
│   └── logs/
│       └── .gitkeep
│
├── .env
├── .env.example
├── .gitignore
├── composer.json
└── README.md
```

6) Khởi tạo project bằng Composer

Week 2 yêu cầu project có `composer.json`, PSR-4 autoload và `vendor/autoload.php` để không phải `require_once` class thủ công nữa.

Bước 1 — tạo thư mục project

```
mkdir php-mini-workshop
cd php-mini-workshop
mkdir public src views config storage
mkdir src/Controllers src/Data src/Support
mkdir storage/logs
```

Bước 2 — khởi tạo Composer project

```
composer init --name="student/php-mini-workshop" --type="project"
--autoload="App\\:src/" --no-interaction
```

Bước 3 — cài package dotenv

```
composer require vlucas/phpdotenv
```

Bước 4 — sinh autoload

```
composer dump-autoload
```

File `composer.json`

```
{
  "name": "student/php-mini-workshop",
  "description": "Week 2 lab - PHP Mini Workshop Registration App",
  "type": "project",
  "autoload": {
    "psr-4": {
      "App\\": "src/"
    }
  },
  "require": {
    "vlucas/phpdotenv": "^5.6"
  }
}
```

7) Tạo .gitignore, .env, .env.example

Week 2 yêu cầu có `.env`, `.env.example` và nhấn mạnh không nên commit `.env` thật vào Git.

`.gitignore`

```
/vendor/
.env
/storage/logs/*.log
.DS_Store
Thumbs.db
```

`.env`

```
APP_NAME=Mini Workshop Registration
APP_ENV=dev
APP_DEBUG=true
APP_URL=http://localhost:8000
ORGANIZER_NAME=VTCA Training Center
MAX_REGISTRATIONS_PER_REQUEST=3
```

`.env.example`

```
APP_NAME=Mini Workshop Registration
APP_ENV=dev
APP_DEBUG=true
APP_URL=http://localhost:8000
ORGANIZER_NAME=
MAX_REGISTRATIONS_PER_REQUEST=3
```

8) Tạo file config tập trung

src/Support/Env.php

```
<?php

namespace App\Support;

class Env
{
    public static function get(string $key, mixed $default = null): mixed
    {
        return array_key_exists($key, $_ENV) ? $_ENV[$key] : $default;
    }

    public static function bool(string $key, bool $default = false): bool
    {
        $value = self::get($key);

        if ($value === null) {
            return $default;
        }

        return filter_var($value, FILTER_VALIDATE_BOOLEAN,
FILTER_NULL_ON_FAILURE) ?? $default;
    }

    public static function int(string $key, int $default = 0): int
    {
        return (int) self::get($key, $default);
    }
}
```

config/app.php

```
<?php

use App\Support\Env;

return [
    'app' => [
        'name' => Env::get('APP_NAME', 'My App'),
        'env' => Env::get('APP_ENV', 'prod'),
        'debug' => Env::bool('APP_DEBUG', false),
        'url' => Env::get('APP_URL', 'http://localhost:8000'),
        'organizer' => Env::get('ORGANIZER_NAME', 'Training Center'),
        'max_registrations_per_request' =>
Env::int('MAX_REGISTRATIONS_PER_REQUEST', 1),
    ],
];
```

9) Tạo dữ liệu mẫu

Lab01 dùng mảng PHP để lưu dữ liệu record. Lab02 vẫn giữ cách này để học viên tập trung vào HTTP và cấu trúc dự án thay vì database.

src/Data/events.php

```
<?php

return [
    [
```

```
        'id' => 1,
        'title' => 'PHP for Beginners',
        'trainer' => 'Anna Tran',
        'date' => '2026-05-05',
        'seats_total' => 30,
        'seats_available' => 12
    ],
    [
        'id' => 2,
        'title' => 'HTTP Basics for Web App',
        'trainer' => 'Minh Nguyen',
        'date' => '2026-05-08',
        'seats_total' => 25,
        'seats_available' => 0
    ],
    [
        'id' => 3,
        'title' => 'Composer and Git Workflow',
        'trainer' => 'Lan Pham',
        'date' => '2026-05-10',
        'seats_total' => 20,
        'seats_available' => 5
    ]
];
```

10) Tạo helper trả response

Week 2 nhấn mạnh đọc method, body, headers trong PHP và trả response bằng `header()` + `http_response_code()` đúng chuẩn.

`src/Support/Response.php`

```
<?php

namespace App\Support;

class Response
{
    public static function json(int $status, array $data = [], array $headers =
[]): void
    {
        http_response_code($status);
        header('Content-Type: application/json; charset=UTF-8');

        foreach ($headers as $name => $value) {
            header($name . ': ' . $value);
        }

        if ($status === 204) {
            exit;
        }

        echo json_encode($data, JSON_UNESCAPED_UNICODE | JSON_PRETTY_PRINT);
        exit;
    }
}
```

11) Tạo controller

src/Controllers/HomeController.php

```php
<?php

namespace App\Controllers;

class HomeController
{
    public function index(array $config, array $events): array
    {
        return [
            'title' => 'Mini Workshop Registration App',
            'app_name' => $config['app']['name'],
            'organizer' => $config['app']['organizer'],
            'app_env' => $config['app']['env'],
            'app_debug' => $config['app']['debug'] ? 'true' : 'false',
            'events' => $events
        ];
    }
}
```

src/Controllers/EventController.php

```php
<?php

namespace App\Controllers;

use App\Support\Response;

class EventController
{
    public function index(array $events): void
    {
        Response::json(200, [
            'message' => 'Workshop list',
            'data' => $events
        ]);
    }

    public function head(): void
    {
        http_response_code(200);
        header('Content-Type: application/json; charset=UTF-8');
        exit;
    }
}
```

src/Controllers/RegistrationController.php

```php
<?php

namespace App\Controllers;

use App\Support\Response;

class RegistrationController
{
    public function store(array $events, array $config): void
    {
        $headers = function_exists('getallheaders') ? getallheaders() : [];
```

```
        $contentType = $headers['Content-Type'] ?? $headers['content-type'] ??
($_SERVER['CONTENT_TYPE'] ?? '');

        if (!str_contains(strtolower($contentType), 'application/json')) {
            Response::json(415, [
                'error' => 'Unsupported Media Type',
                'message' => 'Content-Type must be application/json'
            ]);
        }

        $raw = file_get_contents('php://input');
        $payload = json_decode($raw, true);

        if (!is_array($payload)) {
            Response::json(400, [
                'error' => 'Bad Request',
                'message' => 'Invalid JSON body'
            ]);
        }

        $eventId = $payload['event_id'] ?? null;
        $studentName = trim($payload['student_name'] ?? '');
        $email = trim($payload['email'] ?? '');
        $quantity = (int) ($payload['quantity'] ?? 0);

        if (!$eventId || $studentName === '' || $email === '' || $quantity <=
0) {
            Response::json(422, [
                'error' => 'Unprocessable Content',
                'message' => 'event_id, student_name, email, quantity are
required and must be valid'
            ]);
        }

        if ($quantity > $config['app']['max_registrations_per_request']) {
            Response::json(422, [
                'error' => 'Unprocessable Content',
                'message' => 'Quantity exceeds allowed registration limit per
request'
            ]);
        }

        $selectedEvent = null;
        foreach ($events as $event) {
            if ($event['id'] === (int) $eventId) {
                $selectedEvent = $event;
                break;
            }
        }

        if (!$selectedEvent) {
            Response::json(422, [
                'error' => 'Unprocessable Content',
                'message' => 'Selected event does not exist'
            ]);
        }
```

```
        if ($selectedEvent['seats_available'] < $quantity) {
            Response::json(422, [
                'error' => 'Unprocessable Content',
                'message' => 'Not enough seats available'
            ]);
        }

        $registrationId = time();

        Response::json(201, [
            'message' => 'Registration created successfully',
            'data' => [
                'registration_id' => $registrationId,
                'student_name' => $studentName,
                'email' => $email,
                'event_id' => (int) $eventId,
                'quantity' => $quantity
            ]
        ], [
            'Location' => '/registrations/' . $registrationId
        ]);
    }

    public function options(): void
    {
        http_response_code(204);
        header('Allow: POST, OPTIONS');
        exit;
    }
}
```

12) Tạo view trang chủ

views/home.php

```
<?php /** @var array $data */ ?>
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <title><?= htmlspecialchars($data['title']) ?></title>
</head>
<body>
    <h1><?= htmlspecialchars($data['title']) ?></h1>

    <ul>
        <li>APP_NAME: <?= htmlspecialchars($data['app_name']) ?></li>
        <li>ORGANIZER: <?= htmlspecialchars($data['organizer']) ?></li>
        <li>APP_ENV: <?= htmlspecialchars($data['app_env']) ?></li>
        <li>APP_DEBUG: <?= htmlspecialchars($data['app_debug']) ?></li>
    </ul>

    <h2>Workshop List</h2>
    <?php foreach ($data['events'] as $event): ?>
        <div style="margin-bottom: 16px; padding: 12px; border: 1px solid
#ccc;">
            <p><strong>Title:</strong> <?= htmlspecialchars($event['title'])
?></p>
```

```
            <p><strong>Trainer:</strong> <?=
htmlspecialchars($event['trainer']) ?></p>
            <p><strong>Date:</strong> <?= htmlspecialchars($event['date'])
?></p>
            <p><strong>Seats Total:</strong> <?=
htmlspecialchars((string)$event['seats_total']) ?></p>
            <p><strong>Seats Available:</strong> <?=
htmlspecialchars((string)$event['seats_available']) ?></p>
            <p><strong>Status:</strong> <?= $event['seats_available'] > 0 ?
'Open' : 'Full' ?></p>
        </div>
    <?php endforeach; ?>

    <h2>API Endpoints</h2>
    <ul>
        <li>GET /events</li>
        <li>HEAD /events</li>
        <li>POST /registrations</li>
        <li>OPTIONS /registrations</li>
    </ul>
</body>
</html>
```

13) Tạo bootstrap + router tại public/index.php

Week 2 mini-lab yêu cầu rõ `public/index.php` là bootstrap chính, load autoload, load `.env`, validate biến môi trường, phân biệt dev/prod và route request.

`public/index.php`

```
<?php

require __DIR__ . '/../vendor/autoload.php';

use App\Controllers\EventController;
use App\Controllers\HomeController;
use App\Controllers\RegistrationController;
use App\Support\Env;
use App\Support\Response;
use Dotenv\Dotenv;

$dotenv = Dotenv::createImmutable(dirname(__DIR__));
$dotenv->safeLoad();

$dotenv->required(['APP_NAME', 'APP_ENV', 'APP_DEBUG', 'APP_URL',
'ORGANIZER_NAME', 'MAX_REGISTRATIONS_PER_REQUEST']);
$dotenv->required('APP_DEBUG')->isBoolean();
$dotenv->required('MAX_REGISTRATIONS_PER_REQUEST')->isInteger();

error_reporting(E_ALL);

if (Env::get('APP_ENV', 'prod') === 'dev' && Env::bool('APP_DEBUG', false)) {
    ini_set('display_errors', '1');
    ini_set('display_startup_errors', '1');
    ini_set('log_errors', '1');
} else {
    ini_set('display_errors', '0');
    ini_set('display_startup_errors', '0');
```

```
    ini_set('log_errors', '1');
    ini_set('error_log', dirname(__DIR__) . '/storage/logs/php-error.log');
}

$config = require dirname(__DIR__) . '/config/app.php';
$events = require dirname(__DIR__) . '/src/Data/events.php';

$method = $_SERVER['REQUEST_METHOD'] ?? 'GET';
$path = parse_url($_SERVER['REQUEST_URI'] ?? '/', PHP_URL_PATH);

if ($path === '/' && $method === 'GET') {
    $controller = new HomeController();
    $data = $controller->index($config, $events);
    require dirname(__DIR__) . '/views/home.php';
    exit;
}

if ($path === '/events' && $method === 'GET') {
    (new EventController())->index($events);
}

if ($path === '/events' && $method === 'HEAD') {
    (new EventController())->head();
}

if ($path === '/events' && !in_array($method, ['GET', 'HEAD'], true)) {
    Response::json(405, [
        'error' => 'Method Not Allowed'
    ], [
        'Allow' => 'GET, HEAD'
    ]);
}

if ($path === '/registrations' && $method === 'POST') {
    (new RegistrationController())->store($events, $config);
}

if ($path === '/registrations' && $method === 'OPTIONS') {
    (new RegistrationController())->options();
}

if ($path === '/registrations' && !in_array($method, ['POST', 'OPTIONS'],
true)) {
    Response::json(405, [
        'error' => 'Method Not Allowed'
    ], [
        'Allow' => 'POST, OPTIONS'
    ]);
}

if ($path === '/health' && $method === 'GET') {
    Response::json(200, [
        'status' => 'ok',
        'app' => $config['app']['name']
    ]);
}
```

```
Response::json(404, [
    'error' => 'Not Found'
]);
```

14) Tạo README

`README.md`

```
# PHP Mini Workshop Registration

Week 2 PHP Lab

## Run
php -S localhost:8000 -t public

## Endpoints
- GET /
- GET /events
- HEAD /events
- POST /registrations
- OPTIONS /registrations
- GET /health
```

15) Chạy chương trình bằng web server local

Lab01 và Week 2 đều dùng built-in web server của PHP để chạy local project trong thư mục `public`.

Trong terminal của VS Code, chạy:

```
php -S localhost:8000 -t public
```

Mở trình duyệt:

```
http://localhost:8000
```

Kết quả mong đợi

- Trang `/` hiển thị thông tin app và danh sách workshop
- `GET /events` trả JSON danh sách workshop
- `HEAD /events` trả header/status nhưng không cần body
- `POST /registrations` trả `201 Created` khi thành công
- `OPTIONS /registrations` trả `204 No Content`
- Gọi sai method thì trả `405` kèm `Allow`
- Gửi sai content type thì trả `415`
- Thiếu field hay hết chỗ thì trả `422`

Các quy tắc này bám đúng Week 2 HTTP basics: GET là lấy dữ liệu, POST tạo mới nên ưu tiên 201 + Location, 405 phải có Allow, 415 cho sai media type, 422 cho lỗi nội dung/nghiệp vụ.

16) Test API

Test 1 — GET workshop list

```
curl http://localhost:8000/events
```

Kỳ vọng: `200 OK`

Test 2 — HEAD workshop list

```
curl -I http://localhost:8000/events
```

Kỳ vọng: `200 OK`, không cần body

Test 3 — POST đăng ký thành công

```
curl -X POST http://localhost:8000/registrations \
  -H "Content-Type: application/json" \
  -d '{"event_id":1,"student_name":"Nguyen Van
A","email":"a@example.com","quantity":1}'
```

Kỳ vọng:

- `201 Created`
- có header `Location`
- body JSON báo tạo thành công

Test 4 — POST sai content type

```
curl -X POST http://localhost:8000/registrations \
  -H "Content-Type: text/plain" \
  -d 'hello'
```

Kỳ vọng: `415 Unsupported Media Type`

Test 5 — POST thiếu dữ liệu

```
curl -X POST http://localhost:8000/registrations \
  -H "Content-Type: application/json" \
  -d '{"event_id":1,"student_name":"","email":"","quantity":0}'
```

Kỳ vọng: `422 Unprocessable Content`

Test 6 — Sai method

```
curl -X PUT http://localhost:8000/registrations
```

Kỳ vọng:

- `405 Method Not Allowed`
- header `Allow: POST, OPTIONS`

Week 2 HTTP doc nhấn mạnh rất rõ sự khác nhau giữa 404 và 405, và khi trả 405 cần có `Allow`.

17) Git và GitHub

Lab01 đã dùng quy trình Git cơ bản `init → add → commit → remote → push`. Lab02 giữ nguyên quy trình đó.

Bước 1 — khởi tạo Git

```
git init
git status
```

Bước 2 — commit lần đầu

```
git add .
git commit -m "init: bootstrap php mini workshop project"
```

Bước 3 — commit sau khi hoàn thành route và API

```
git add .
git commit -m "feat: add events endpoint and registration api"
```

Bước 4 — commit sau khi hoàn thiện test và README

```
git add .
git commit -m "docs: add readme and api test instructions"
```

Bước 5 — đưa lên GitHub

```
git branch -M main
git remote add origin https://github.com/USERNAME/php-mini-workshop.git
git push -u origin main
```